



Player Documentation

- [Player Methods](#)
- [Console Commands](#)

All player methods are related to setting particular values about the user and their inventory.

Player Methods

```
player.sync()
```

Method

Saves any modified and persistent player attribute values to disk. Shouldn't be called too frequently to avoid thrashing the SD card. If you make a page that uses the player class and you set attribute values that should be persistent, call this in the `remove()` function of your app to flush them to disk.

`player.getinfo()`

Method — Returns: `object`

Returns all player attributes that are in the state. At minimum this includes:

Attribute	Description
<code>name</code>	Player name
<code>level</code>	Current level

Attribute	Description
xpNext	Amount of XP needed to reach the next level
maxHP	Maximum hit points
maxAP	Maximum action points
maxWg	Maximum carry weight
hp	Current hit points (determined by the sum of limb conditions)
wg	Current carry weight (weight of inventory)

Also returns any other values set against player attribute values.

`player.resetinventory()`

Method

Resets all inventory items back to their default inventory state for both Fallout 3 and New Vegas game modes.

`player.calculateInvWeight()`

Method

Recalculates the player `wg` value — the total mass of all items in their inventories. Users probably won't need this.

Console Commands

Methods that mirror Fallout 3 console commands.

`player.getav(av)`

Method

Takes a string `av`, the attribute value name, and returns the value of that key from the player object.

Parameters

Name	Type	Default	Description
<code>av</code>	<code>string</code>	—	The attribute value name to look up from the player object

Example

```
const name = player.getav('name');  
// name = "Albert"  
  
const karma = player.getav('karma');  
// karma = 250
```

player.setav(av, value, persist)

Method

The set version of `player.getav()`. Takes an attribute value name and the value to set it to.

The optional third parameter `persist` is non-standard (not in the game commands). If set to `true`, the value will be persisted to disk when `player.sync()` is called. Defaults to `false`. It's best not to persist unless you really need something to survive a reboot.

Parameters

Name	Type	Default	Description
<code>av</code>	<code>string</code>	—	The attribute value name

Name	Type	Default	Description
			on the player object
<code>value</code>	<code>any</code>	—	The value to set
<code>persist</code>	<code>boolean</code>	<code>false</code>	If <code>true</code> , persists this value to disk on <code>player.sync()</code>

`player.addperk(formId)`

Method

Adds a perk by its form ID. The `formId` is a numeric value and can be passed as hex to match game conventions.

Which perk IDs are valid depends on the current game mode of the Pip-Boy. If it's in New Vegas mode, it'll take New Vegas perk IDs and add them to a New Vegas specific version of `PERKS.JSON`. Both `player.addperk()` and `player.removeperk()` modify values in mode-specific `PERKS.JSON` files.

Parameters

Name	Type	Default	Description
<code>formId</code>	<code>number</code>	—	The perk's form ID (e.g. <code>0x00031DC4</code> for "Computer Whiz")

Example

```
player.addperk(0x00031DC4);
```

```
player.removeperk(formId)
```


Removes a perk by its form ID. The same as `player.addperk()` but removes the perk instead of adding it.

Which perk IDs are valid depends on the current game mode of the Pip-Boy. If it's in New Vegas mode, it'll use New Vegas perk IDs from a New Vegas specific version of `PERKS.JSON`. Both methods modify values in mode-specific `PERKS.JSON` files.

Parameters

Name	Type	Default	Description
<code>formId</code>	<code>number</code>	—	The perk's form ID (e.g. <code>0x00031DC4</code> for "Computer Whiz")

Example

```
player.removeperk(0x00031DC4);
```

player.advlevel()

Method

Advances the player level by 1. Takes no parameters.

player.setlevel(level)

Method

Sets the player's level to the given number.

Parameters

Name	Type	Default	Description
level	number	—	The level to set player to

player.addItem(formId, count)

Method

Adds items to the player's inventory by form ID and count. Which form IDs are valid depends on the current game mode of the Pip-Boy (e.g. Fallout 3 vs New Vegas).

Parameters

Name	Type	Default	Description
formId	number	—	The item's form ID (e.g. 0x0001519E for Nuka-Cola)
count	number	—	The number of items to add

Example

```
player.additem(0x0001519E, 6);
```

player.additemhealthpercent(formId, count, condition)

Method

Same as `player.additem()` but also takes a condition percentage for the items. Which form IDs are valid depends on the current game mode of the Pip-Boy.

Parameters

Name	Type	Default	Description
formId	number	—	The item's form ID
count	number	—	The number of items to add.

Name	Type	Default	Description
condition	number	—	Item condition as a percentage (0 – 100)

Example

```
player.additemhealthpercent(0x0001519E, 6, 75);
```

```
player.removeitem(formId,  
count)
```

Method

Removes items from the player's inventory by form ID and count.

 **Not yet implemented.**

Parameters

Name	Type	Default	Description
formId	number	—	The item's form ID
count	number	—	The number of items to remove

`player.equipitem(formId)`

Method

Equips an item by its form ID.

 **Not yet implemented.**

Parameters

Name	Type	Default	Description
<code>formId</code>	<code>number</code>	—	The item's form ID